→ Source documents is nothing but files, pdf

→ chunking is the process of breaking down of bigger documents into smaller chunks
↳ like paragraphs

→ Em is not as LLM. converts the english words to mathematical or vector representation according to their respective chunks

Ex:- In the flowchart 3-3.

Vector Embeddings Ex:-

	x	y	z
cat -	[34,	8,	7.5]
kitten -	[33,	8,	2]
Elephant -	[2,	62,	2]
Dog -	[47,	8,	2]

} looking those both are similar.
} but these not.

Each num in the vector embedding is a dimension

Popular Embedding Models: More are there.

Open AI (most popular choice):

- text embedding - 3 small : 1536 dimensions (default)
  Great for most use cases (can reduce to 512, 1024 or any size ≤ 1536)
- text embedding - 3 - large - 3072 dimensions (default) -
  Best performance, but costs more (can reduce to: 256, 512, 1024, 1536, or any size ≤ 1536)

Both reduce dimensions to save costs

Why the conversion of mathematic Representation is done?

seen in 2nd part (Retrieval Pipeline)

RAG - Retrieval Augmented

- It is a method where we combine LLMs with a retrieval system.

LLMs have a context window (limited no of information).

TO avoid the limited No of tokens used in LLMs we use RAG.

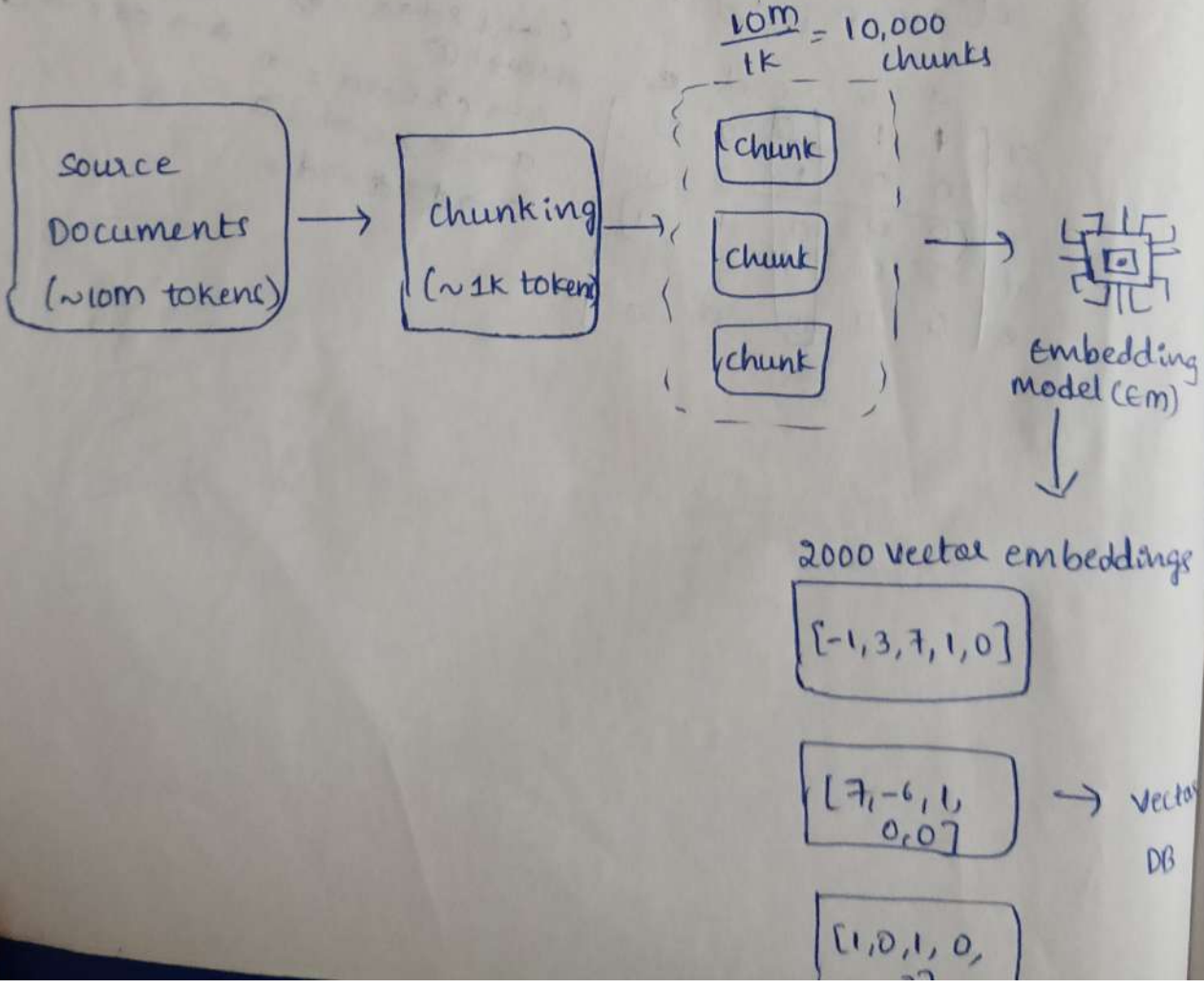
tokens - one character or one word - 1unit of text

Ex: "Hello" is 1 token.

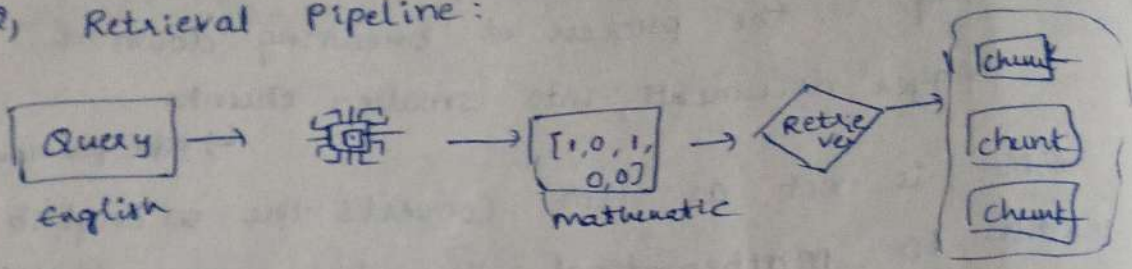
"I'm" are 2 tokens

Two parts of RAG:

1. Knowledge-base construction (Ingestion pipeline):



2) Retrieval Pipeline:



Retriever is connected to vector DB of ingestion pipeline

Chunk 1 Text
" 2 "
" 3 "
Question

You must use the same embedding model for both your documents AND user queries

Ingestion pipeline:

1. loading the files
2. Chunking the files
3. Embedding and storing in vector DB

loading source documents → loader.load() = doc

displays the contents

text_splitter = CharacterTextSplitter(

{

}

chunks = text_splitter.split_documents(doc)

Vectorstore = Create vectore store (chunks)

{Persist directory} - openAI Embedding model

Retrieval pipeline:

Cosine algorithm is used for Retrieval matching

Recreates another model

Retriever does the top K chunks with highest similarity scores to the users query embedding

retriever.invoke(query)

TO confirms that they give best chunks are not - ASK chatgpt

Cosine similarity:

Reason why we are able to fetch the Matching chunks for a user query. from vectorDB

- Angle blw vectors
- Range from 0 to 1

$$\text{cosine_similarity} = (A \cdot B) / (\|A\| \times \|B\|)$$

$A \cdot B$ - dot product of vector A & B

$\|A\|$ = magnitude (length) of vector A

$\|B\|$ = " " B

$\|A\| + \|B\| \Rightarrow$ is exactly 1.

so simplifies to Modern embedding models (open AI's text embedding 3 sma

cosine_similarity = A . B (Dot product)

History aware generation :

The key difference :

In basic RAG, each query is treated independently The retriever takes your exact question and searches for chunks.

In history-aware RAG, there's one crucial extra step : query reformulation

Before searching the system looks at the conversation history and rewrites vague or context-dependent questions into clear, standalone questions

Why this matters - follow up questions.

Humans naturally ask follow up questions using pronouns, references and assumptions based on previous conversation

Introduction to RAG chunking :

Rag Ingestion Pipeline :

- Document loading (PDF, Docx, Txt → text)
- Text chunking (long text → smaller pieces)
- Embedding (text chunks → vectors)
- storage (vectors → vector database)

Chunking determines how you context gets divided for retrieval

Our RAG system doesn't search entire documents It searches chunks

Problems with Basic chunking:

Simple, but crude

- splits at mid sentence
- Breaks related concepts apart
- context gets lost across chunks.
- Poor retrieval quality.

Bad chunks = Bad answers

5 better chunking strategies:

1. Character Text splitter: (Beyond basic chunk-size)
   - custom separators (split on specific patterns)
   - useful for simple, uniform documents.
2. Recursive Character Textsplitter (upgrade from character text splitter)
   - split paragraphs, sentences, words
   - fails if chunks are too big.
3. Document-specific splitting (Respects document structure)

   PDF: splits by pages, sections

4. Markdown: splits by headers, codeblocks, lists
4. Semantic Splitting (content-aware boundaries)
   - splits when meaning changes, not just by size
   - Keep related concepts togethers
   - More intelligent but expensive.
5. Agentic splitting (AI-powered chunking)

   LLM analyzes content and decides optimal splits

   most expensive becoz LLM is involved.

Character Text splitter :-

- It doesn't just cut at character limits. It follows a split-first, merge second approach.

Split : Break text at separators

Merge : combine pieces until hitting chunk-size limit

Chunk size : let's say 100 characters

→ split the 100 characters of each chunk and put \n\n as after each chunk.

- the problem : No of characters in a single piece is > 100 . then we cannot split. No double new (\n\n) line in it

decreasing order of priority of separator :

["\n\n", "\n", ".", " ", ""]

Recursive character Text splitter :

- Recursively multiple separators in this

Semantic chunking :

- used in Embedding models

breaks up long documents into meaningful pieces

3 steps

1. Encode : Convert each sentence into embeddings
2. compare : calculate similarity scores nearby sentence
3. Split : Create boundaries where similarity drops

AI Agentic splitting:

- Reasoning ability of LLM's we see in this
- Initialise LLM
sample text

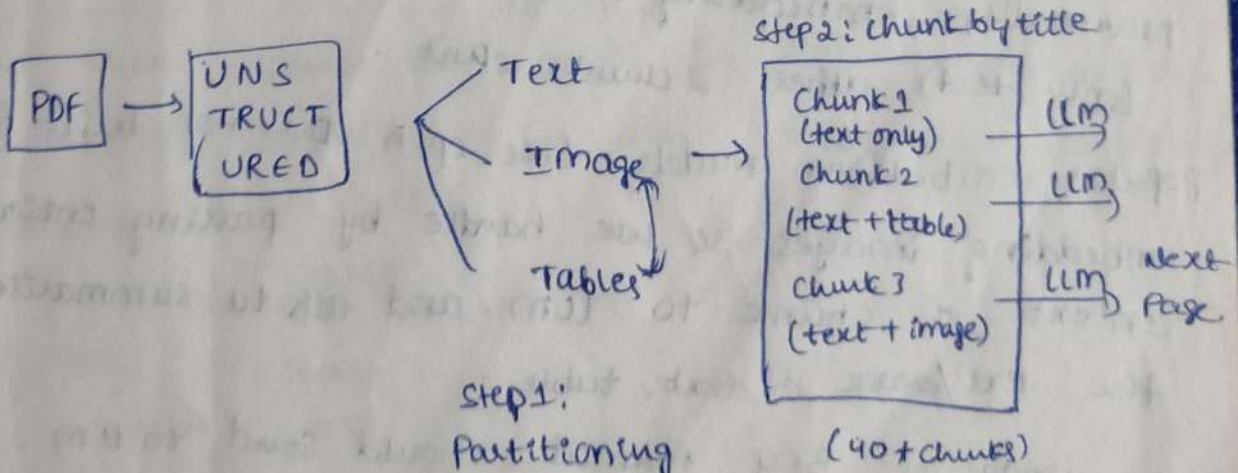
drawback: Not commercially viable for building a product having complex PDF's with images complex files ⇒ these 4 chunking methods are not ex preferable.

for that popular tools are there ETL+ for GenAI data - Transform complex, unstructured data into clean, structured data securely continously. Effortlessly.

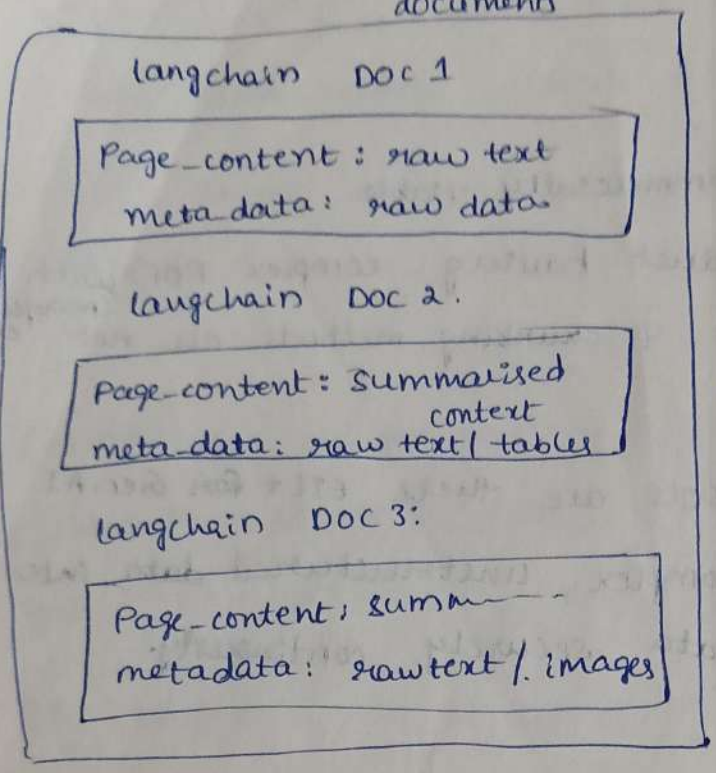
Multi Model RAG with images and documents:

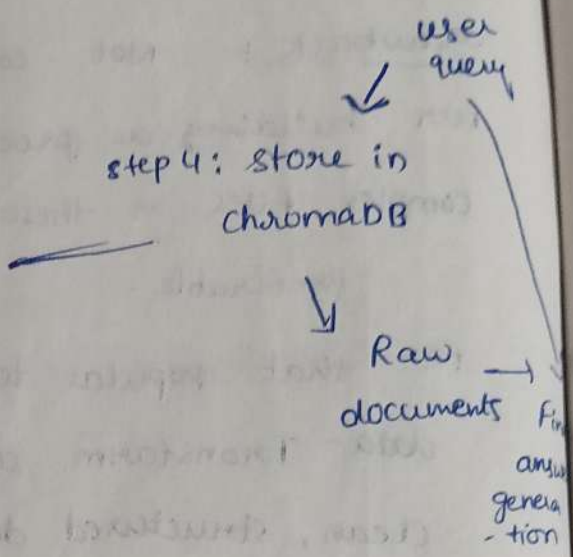
- Hybrid model: chunks that have table or image. summarise it and convert to vector embeddings

If chunk has only text - we need not to summarise we can easily embed raw data.



Step3: Summarising chunks and converting it into langchain documents





Atomic elements : Every single atomic element is unstructured, it has a type like table, text, etc. header, image

Chunk by title : clubbing things 1st, 2nd, -- narrative text untill it reach next title. those all narrative texts are considered to be one chunk.

Normally embedding model can only embed chunk 1 (only text) other 2 chunks can't.

Popular embedding models like open AI's not built for embedding images. so we handle by passing entire content of chunk to LLM, and ask to summarise the text, image & text, table.

image as B64 format should send to LLM and ask to give the questions related to that chunk what may user ask in feature.

langchain models documents is simply a class.
in that we have two attributes
- Page content : main content of document.
- Metadata : Page num, where the chunk is retrieved

chromaDB : vector store we can locally host it.
it can store all langchain documents.

This is the ingestion pipeline.

user asks the question then the chromaDB is going to fetch the top five chunks that have most similar semantic meaning.

to_dict() methods gives different attributes in a class

Every single atomic elements gives the element_ID, text, type

element [50].todict()

```
tables = [element for element in elements if element.
                    category == 'Table']
print(f"found {len(tables)} tables")
tables[0].to_dict()
```

gives the no of tables in the PDF ⇒ elementID, text, title --
same you may give for images.

---

and then separate the chunks by title. and use the LLM and then summarize it and convert into langchain documents.

HTML for table text version
B64 " image "
raw_text " text "

NO of chunks = NO of langchain documents.

Advanced Retrieval Technique:

① Simple Similarity Search:

Intialise retreiver, K=3 returns top 3 chunks

② Similarity with score threshold:

Set the type and add new property called "score_threshold" = 0.3

① has drawback. ⇒ for a query how many? It also gives 3 chunks Not correct

for that score_threshold is to be set

③ Maximum Marginal Relevance (MMR):

- Balances relevance and diversity
- Avoids redundant (repeated) results.
- Its speed is slower

Uses two step process:

i) Find chunks relevant to your query

ii) Among these relevant chunks, pick most diverse one

For increase the accuracy of RAG - Multi-Query RAG Retrieval:

- Generate Multiple Query variations
- Search with each query variations and store results
- store results for RRF (Reciprocal Rank fusion)

RRF : simple but effective.

Method that combines multiple ranked chunks lists by giving each chunk a score based on its positions across all lists.

RRF_score = Σ (1/(k + rank_position))

k - constant. (assumed to be 60)

rank_position - position of x chunk in query result.

Drawback:

We are treating 1% similarity difference as if one document is twice as good as the other.

So that's why k is fixed as 60 for every single chunk.

Hybrid search :

Combination of two different search strategies

Vector search, classical older x keyword search strategy

drawbacks of vector search / Dense :-

- can miss important keywords

keyword approach / sparse Retreival : BM25 Retriever (Best matching)

- looks exact word matches in your chunks
- super reliable for finding terms, names, model numbers

We needed both

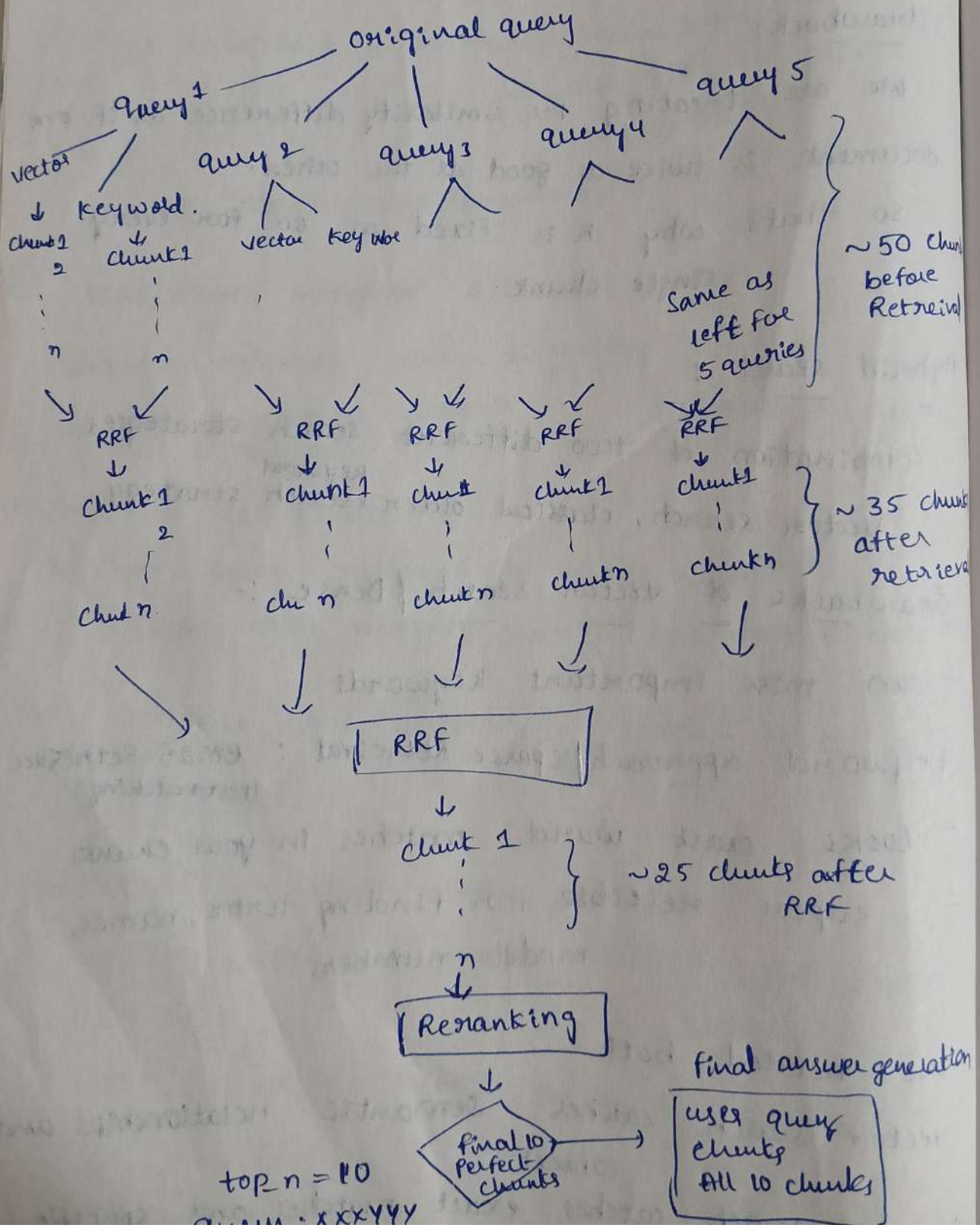
Vector search catches semantic relationships and context

keyword search catches exact matches and specific terminology.

Reranking:-

1. User query → Multiple Retrievers → Individual Rankings → RRF fusion → final Ranking → Generation

Reranker :



ndividual

ng →

~50 Chunks before Retrieval

35 chunks after retrieval

uation

is a specialised AI model

acts as "quality inspector"

2 stages of RAG Pipeline:

i) Embeddings (fast & Broad):

Query → Vector → Compare with 1M chunks → Top 100 chunks.

It can be fast, cheap, Releible but it's just an approximation

ii) Reranker (precise & focussed):

Query + Each of 25 chunks → Reranker / Cross-encoder → precise relevance scores

It can be precise, context-aware, expensive.

Increase probability that top 10 are best

Embedding (Bi-encoder approach): cosine similarity

In the Encoding happens separately for query and chunk. so that's the problem, model never sees the query and chunk together.

Reranker (cross-encoder / joint processing approach):

combines both query and chunk together with SEP keyword.

and understands their relationship.

It is more accurate than embeddings.

We need it when we have more chunks and we need to filter it otherwise directly send to LLM to generate final answer.